**USED MEDIAS LLC**
EMBEDDED SYSTEMS DIVISION

# Engineering Documentation

nat-os

| DOCUMENT | REVISION | STATUS | CLASSIFICATION |
|---|---|---|---|
| nat-os | — | — | Internal |

Hardware: developed and verified on the ESP32-2432S028R ("Cheap Yellow Display")

> **Renamed from `cyd-os`.** Document numbering and section numbering are unchanged — `UM-CYDOS-014 §5.2` in an older commit message is `UM-NATOS-014 §5.2` here. The rename reflects that only the pin maps and the panel and touch drivers are specific to the Cheap Yellow Display; everything above them assumes an ESP32 and nothing more.

## Purpose of this set

These reports record the design of nat-os in enough detail that the project can be picked up cold — by someone else, or by the author after six months away — without re-deriving decisions from the source.

Each report states **what was decided, what it was measured against, and what remains unverified**. Claims that were tested on hardware are marked as such. Claims taken from documentation or reasoning are marked separately, because on a from-scratch kernel the difference between "this is true" and "this should be true" is the difference between a working boot and a silent reboot.

## Index

| ID | Title | Covers |
|---|---|---|
| UM-NATOS-013 | Milestone 5 Verification Report | Application table and lifecycle, three-level scheduling, the shell, the escape attempt, and messaging without shared memory |
| UM-NATOS-014 | Locking Primitives | Critical sections vs blocking mutex, task blocking and idle, a measured starvation defect, and why contention cost is the number of blocking events rather than the time held |
| UM-NATOS-015 | Display Driver | ILI9341, span rendering with no framebuffer, and the path from bit-banged SPI to SPI2 with DMA |
| UM-NATOS-016 | Display Syscalls, and a Total System Freeze | Per-application viewports, pointer discipline, measured containment, and a yield that stopped the clock |
| UM-NATOS-017 | Touchscreen, and a Verification Method That Failed Three Times | XPT2046 over PENIRQ, the GPIO two-bank bug, calibration by controlled input, a capture that erased its own evidence, input confinement, and an axis that was inverted for three months because the direction test read its own worst sample |
| UM-NATOS-018 | Persistence, and a Read Defect That Looked Like the Wrong Thing | SPI flash driver, a checksummed record that survives a power cycle, the inherited clock divider that shifted every read by a bit, and two hypotheses tested against stale firmware |
| UM-NATOS-019 | Failure Handling, and Three Mechanisms That Had Never Fired | Stack guards enforced in the scheduler, the watchdog erasing its own panic reports, measured stack margins, a UART receive path one byte behind, and a fault reported to flash and to the panel |

## Reading order

New to the project: **001** → **002** → **004** → **003**. That gives the shape of the system, how it starts, where things live, and why the calling convention is unusual.

Picking up implementation work: **006** (what is known-good) then **007** (what is next and what it will break).

Reproducing a build: **005** alone is sufficient.

## Status summary as of this revision

| Item | State |
|---|---|
| Milestone 0 — kernel boots, self-checks pass | **Complete, verified on hardware** |
| Milestone 1 — timer interrupt, tick counter | **Complete, verified on hardware** |
| Milestone 2 — native task switching | **Complete, verified on hardware** — 3,400+ switches, zero corruption |
| Milestone 3 — heap and arena model | **Complete, verified on hardware** — all three exit criteria |
| Milestone 4 — bytecode interpreter | **Complete, verified on hardware** — program runs, six fault classes contained |
| Isolation | **Live** — every VM memory access bounds-checked, UM-NATOS-012 §6.2 |
| Full stack | **Running** — bytecode hosted in a preemptible native task, 3.3M instructions across 450 preemptions with zero accounting drift, UM-NATOS-012 §6.6 |
| Milestone 5 — multiple applications | **Complete, verified on hardware** — all three exit criteria |
| Roadmap M0–M5 | **Complete.** Six native tasks, three scheduling levels, a shell, and an application deliberately written to escape its arena that could not |
| Locking | **Complete** — critical sections and a blocking mutex; heap and console both arbitrated, UM-NATOS-014 |
| Task blocking | **Live** — `TASK_BLOCKED`, `TASK_SLEEPING` and an idle task using `WAITI`, UM-NATOS-009 §11 |
| Scheduling | **Priorities** — three levels, strict, with `task_sleep()` and priority inheritance; renderer 2 fps → 8.5 fps, UM-NATOS-009 §11 |
| 3D renderer | **Running** — grid raycaster, no framebuffer (measured not to help), UM-NATOS-015 §5.7 |
| Display | **Working on hardware** — ILI9341, no framebuffer, **43 ms** full-screen fill via SPI2 + DMA, UM-NATOS-015 §5.5 |
| Application graphics | **Live** — `FILL` / `TEXT` / `DIMS` confined to per-application viewports; 0 escapes across 136 audited fills, UM-NATOS-016 §5 |
| Touch | **Working on hardware** — XPT2046 gated on PENIRQ **and** pressure; X axis was inverted for three months and is now calibrated from four labelled corners, UM-NATOS-017 §4.1, §7.1 |
| Application input | **Live** — `SYS TOUCH` confined to the asking application's viewport; 81 delivered, 109,211 withheld, 0 confinement failures, UM-NATOS-017 §8 |
| Application messaging | **Live** — copied through a kernel mailbox, never shared memory; 278 sent / 277 delivered / 0 bad buffers, UM-NATOS-013 §8 |

| Item | State |
|---|---|
| Application bitmaps | **Live** — `SYS BLIT`, arena-bounded source and viewport-clipped destination, UM-NATOS-012 §10 |
| DRAM budget | **Measured** — 158,000 B allocatable today (167,680 before `TASK_MAX` rose to 8); a full framebuffer is unnecessary, UM-NATOS-010 §7.2 |
| Flash cache | **Enabled** — `.rodata` mapped from flash, UM-NATOS-011 |
| Persistence | **Live** — checksummed record in a flash sector at 2 MB; boot counter and a cumulative frame count survived 16 resets, UM-NATOS-018 §6 |
| Failure handling | **Enforced** — guards checked on every switch across all eight tasks; `hang` / `fault` / `smash` each trigger their path on demand, UM-NATOS-019 §5 |
| Fault reporting | **Three ways** — flash record read back by the next boot, UART report, and the reason drawn on the panel; ordered by decreasing reliability, UM-NATOS-019 §6–7 |
| Stack margins | **Measured** — worst task uses 444 B of 2,048; minimum margin 78%, UM-NATOS-019 §3.1 |
| Version control | **Initialised 2026-08-14** |
| JTAG debug probe | Ordered, not in hand |
| Bootloader IRAM overlap | **Closed** — investigated and disproved, UM-NATOS-004 §5 |
| Panic handler | **Closed** — exercised deliberately with an `ill` instruction; prints EXCCAUSE/EPC and halts |
| Watchdogs | **Closed** — measured armed, now disabled and read back, UM-NATOS-009 §8 |

**Standing rule for interrupt handlers — clear `PS.EXCM` before calling C.** Hardware sets `EXCM` on interrupt entry, and while it is set the Xtensa zero-overhead loop-back is disabled: a hardware loop body executes once and falls through to whatever sits at `LEND`. GCC emits these loops for ordinary counted C loops, so this silently degrades **every C function reachable from a handler** — drivers and the VM interpreter included, not just the scheduler where it was found. `_handler_level3` now clears it; any future handler must do the same. UM-NATOS-009 §6.

**Standing rule for shared registers — a hardware register with a software shadow has one safe shape: either one writer, or every writer maintains the shadow.** `timer.c` kept `g_next` as its idea of the comparator deadline while `task_yield()` wrote CCOMPARE1 directly. The handler then added a whole interval to a deadline that was only 64 cycles old, so every yield pushed the tick further out — 18 tick periods, 183 ms, before anything noticed. The rule below was obeyed exactly and did not prevent it, because it constrains the WRITER and the defect was in the other party's bookkeeping. UM-NATOS-008 §8.

**Standing rule for the scheduler — a yield must never defer the clock it depends on.** `task_yield()` originally wrote `CCOMPARE1 = ccount + 64` unconditionally, so any loop yielding faster than 64 cycles pushed the deadline ahead of `CCOUNT` forever, the timer interrupt stopped firing, and the whole kernel froze — the tick is what drives every context switch. Any routine adjusting a scheduler deadline must only ever move it **earlier**. UM-NATOS-016 §3.

**Standing rule for verification — when an instrument reports its own reading invalid, that outranks every value printed beside it.** Three times across two sessions a frozen marker, a sample counter stuck at an identical value, and a boot banner in a serial capture each said "this measurement is not what you think", and the plausible-looking numbers next to them were believed anyway. Two conclusions were published and later retracted as a result. Latch the quantity so timing cannot lie about it, then feed the system a controlled input rather than interpreting an uncontrolled one. UM-NATOS-017 §6.

**Standing rule for debugging — a negative result is only informative if the experiment demonstrably ran.** Two flash hypotheses were recorded as tested against a board that had never been reflashed: `build.ps1` builds, but `build.ps1 -Flash` is what flashes, and the invocation used named a script that does not exist. Every hypothesis returned bit-identical output, which was read as "none of these are the cause" when it meant "no experiment has run yet". The signature to watch for is **a run of results that do not vary when the input does** — suspect the harness before the theory, and verify the change reached the target rather than inferring it from the absence of an error. UM-NATOS-018 §5.

**Standing rule for verification — a startup artefact is not evidence the thing it introduces works.** The shell was signed off because its banner printed; the banner proves a task was created and the TRANSMIT path works, and says nothing about receive. The receive path was one byte behind for the shell's entire existence, so pressing Enter did nothing until the next keystroke — and every automated test drove it with CR **and** LF, the one input shape that hides it. Stack guards and the panic handler went unexercised for the same reason: each was confirmed to EXIST rather than observed to WORK. Trigger the mechanism on purpose, or treat it as untested. UM-NATOS-019 §4.1.

**Standing rule for calibration — never infer direction from the endpoint of a gesture.** The first and last samples of a drag are its two least trustworthy, because both sit at a contact transition where the panel is not bridged and the ADC reads its rail. A rail reading is near the top of the range, so a drag ending anywhere "ends near its maximum" and EVERY axis appears to increase — the test can only return one answer. The touch X axis was backwards for three months behind exactly that. Calibrate from labelled points, each a known position paired with a reading, and let direction fall out of comparing labels. UM-NATOS-017 §7.1.

**Standing rule for lock contention — the cost is the NUMBER of blocking events, not the time the lock is held.** Each one costs a scheduling round-trip whether the lock was held for 24 ms or 24 µs: the blocked task is descheduled and must be selected again. Measured here at 63 ms per contention against a 24 ms hold, with the lock free 88% of the time while two tasks sat blocked on it. Narrowing the hold by 25% changed the outcome by nothing. The levers are batching (fewer takes) or not blocking at all ( `try_lock` ) — shortening holds, the intuitive move, does nothing. UM-NATOS-014 §10.5.